使用 pytorch 构建 transformer 模型练习

概述：因本人刚开始学习，暂先对模型基础进行练习，使用 pytorch 进行搭建 transformer 模型，主要内容包括以下：

一环境搭建

1.1 相关库及引入：

```
import torch
import torch.nn as nn
import torch.nn.functional as f
from torch.autograd import Variable
import math
import matplotlib. pyplot as plt
import numpy as np
import copy
```

1.2 生成句子

```
# 关于 word embedding，以序列建模为例
# 输入句子有两个，第一个长度为 2，第二个长度为 4
src_len = torch.tensor([2, 4]).to(torch.int32)
# 目标句子有两个。第一个长度为 4， 第二个长度为 3
tgt_len = torch.tensor([4, 3]).to(torch.int32)
print(src_len)
print(tgt_len)
```

```
tensor([2, 4], dtype=torch.int32)
tensor([4, 3], dtype=torch.int32)
```

用随机数生成句子，用 0 填充空白位置，保持所有句子长度一致

```
src_seq = torch.cat([torch.unsqueeze(F.pad(torch.randint(1, max_num_src_words, (L, )), (0, max(src_len)-L)), 0) for L in src_len])
tgt_seq = torch.cat([torch.unsqueeze(F.pad(torch.randint(1, max_num_tgt_words, (L, )), (0, max(tgt_len)-L)), 0) for L in tgt_len])
print(src_seq)
print(tgt_seq)
```

```
tensor([[2, 5, 0, 0],
        [5, 2, 4, 6]])
tensor([[4, 4, 5, 6],
        [3, 5, 2, 0]])
```

1.3 生成字典

在该字典中，总共有 8 个字(行)，每个字对应 8 维向量(做了简化了的)。注意在实际应用中，应当有几十万个字，每个字可能有 512 个维度.

```
# 构造 word embedding
src_embedding_table = nn.Embedding(9, model_dim)
tgt_embedding_table = nn.Embedding(9, model_dim)
# 输入单词的字典
print(src_embedding_table)
# 目标单词的字典
print(tgt_embedding_table)
```

```
tensor([[-0.8241,  0.7997,  0.4845,  0.6752, -0.9564, -0.1939, -1.0436, -1.2443],
        [ 0.4802,  0.7956, -1.4418,  0.8636,  0.5995,  0.6795,  1.4303, -0.4107],
        [-0.6812,  0.1180,  0.1477,  0.2795, -0.0339,  0.1421,  0.7598,  0.1729],
        [ 0.2985, -0.2763,  0.4994, -1.7550, -0.0841, -1.6392,  1.1468, -0.3015],
        [-1.3102,  1.4121,  0.0382,  1.1614, -1.9609,  0.5185,  0.0494, -0.2884],
        [-0.4421, -0.5209, -1.4645, -0.3028, -0.2654, -1.1694,  1.0671,  0.6909],
        [ 0.5936, -1.1632,  0.8123,  0.2825, -0.5600,  0.8720,  0.6497,  1.2447],
        [ 0.2607,  1.4345,  0.8274, -2.0675,  0.8188,  0.6304, -2.2034,  1.3743],
        [-0.5538, -1.0607, -0.2719, -0.3418,  0.2144, -2.4699,  0.7817,  0.3684]],
       requires_grad=True)
Parameter containing:
tensor([[ 1.1715, -1.1206,  0.4931,  0.6972, -1.4218, -0.3685, -0.0692,  2.0158],
        [ 0.4817,  1.5101, -0.8384,  1.3360, -0.6894,  1.8690,  0.0157,  0.0413],
        [-0.8254, -0.5129, -0.4688,  1.3683,  0.8167,  0.4063,  0.5235,  0.5051],
        [-0.5658,  0.6115, -1.1790, -1.4233,  1.7615, -0.5084, -1.2277, -1.8588],
        [ 0.9158,  0.3662,  0.3328,  0.7405,  1.0675, -0.0801, -1.8032,  0.9099],
        [-0.7435, -2.7788,  2.0797,  0.1555, -0.8106, -1.1135,  1.0354,  0.8993],
        [-0.7986,  0.3977, -1.6849,  0.9865,  1.3671, -0.5937, -0.6682, -0.7601],
        [ 0.5922, -1.4463, -1.5189,  0.2860,  0.6239, -1.9620,  0.6299, -0.6936],
        [-0.7233, -1.7260,  0.4460, -0.0182, -1.1833, -0.0322, -1.9604, -1.6905]],
```

1.4 得到向量化的句子

通过字典取出 1.2 中得到的句子

```
# 得到向量化的句子
src_embedding = src_embedding_table(src_seq)
tgt_embedding = tgt_embedding_table(tgt_seq)
print(src_embedding)
print(tgt_embedding)
```

```
tensor([[[-0.9116,  1.3737,  0.3385,  1.6658,  0.5947,  0.2507,  0.0161,
           0.9145],
         [ 1.1358, -0.3259,  0.2760, -0.2691, -0.3793, -1.3110, -0.7361,
           1.4555],
         [ 0.6999, -0.1037, -0.0252, -1.1971,  0.9864, -0.3915, -1.3199,
          -0.2029],
         [ 0.6999, -0.1037, -0.0252, -1.1971,  0.9864, -0.3915, -1.3199,
          -0.2029]],

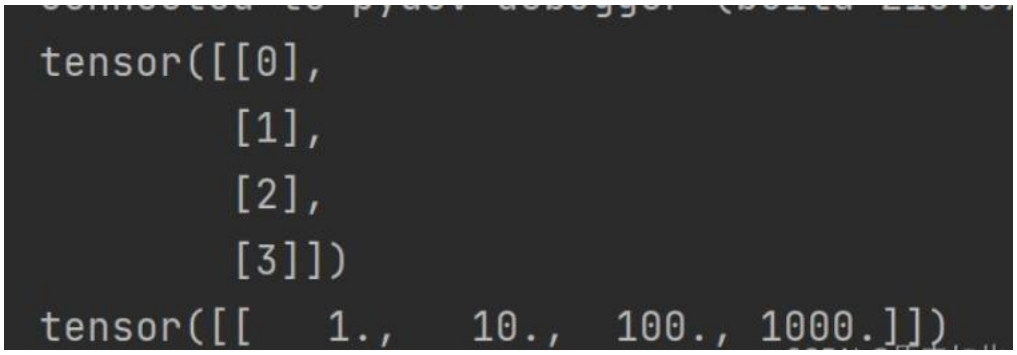
        [[-0.9116,  1.3737,  0.3385,  1.6658,  0.5947,  0.2507,  0.0161,
           0.9145],
         [-0.4729, -0.6720, -0.1409, -0.7971, -0.9514,  0.1150, -0.2459,
          -1.3785],
         [-0.9116,  1.3737,  0.3385,  1.6658,  0.5947,  0.2507,  0.0161,
           0.9145],
         [-0.0892,  0.7491,  0.0372, -0.9656,  1.4412, -0.0440,  2.1741,
          -1.7773]]], grad_fn=<EmbeddingBackward0>)
tensor([[[ 0.7688,  1.6833,  2.3162, -1.1643, -1.5085, -2.1097, -1.7368,
           1.3160],
         [-0.0876, -0.8724,  1.3794,  0.7731,  0.8418,  0.4816, -0.1509,
          -2.2804],
         [ 0.7688,  1.6833,  2.3162, -1.1643, -1.5085, -2.1097, -1.7368,
           1.3160],
         [-0.4685, -0.9799,  1.0073, -0.0323,  0.1959, -1.1352,  2.0116,
          -0.0623]],

        [[-0.0876, -0.8724,  1.3794,  0.7731,  0.8418,  0.4816, -0.1509,
```

二、位置编码

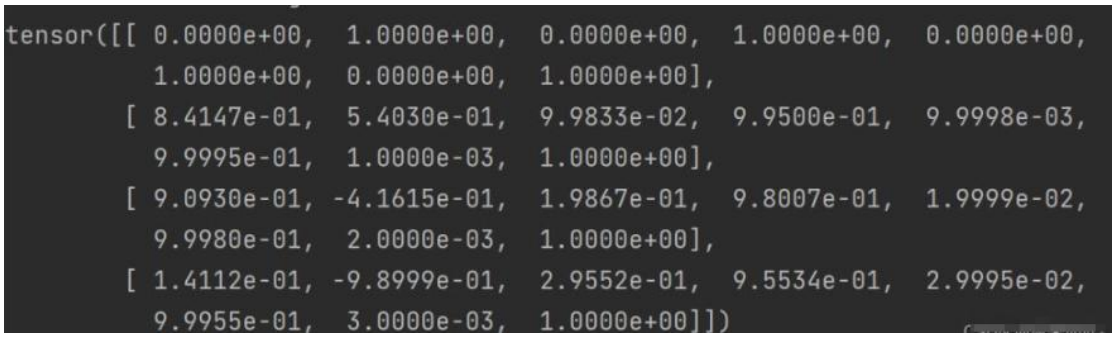
2.1 计算括号内的值

```
# 得到分子 pos 的值
pos_mat = torch.arange(4).reshape((-1, 1))
# 得到分母值
i_mat = torch.pow(10000, torch.arange(0, 8, 2).reshape((1, -1))/8)
print(pos_mat)
print(i_mat)
```



2.2 得到位置编码

```
# 初始化位置编码矩阵
pe_embedding_table = torch.zeros(4, 8)
# 得到偶数行位置编码
pe_embedding_table[:, 0::2] =torch.sin(pos_mat / i_mat)
# 得到奇数行位置编码
pe_embedding_table[:, 1::2] =torch.cos(pos_mat / i_mat)
pe_embedding = nn.Embedding(4, 8)
# 设置位置编码不可更新参数
pe_embedding.weight = nn.Parameter(pe_embedding_table, requires_grad=False)
print(pe_embedding.weight)
```

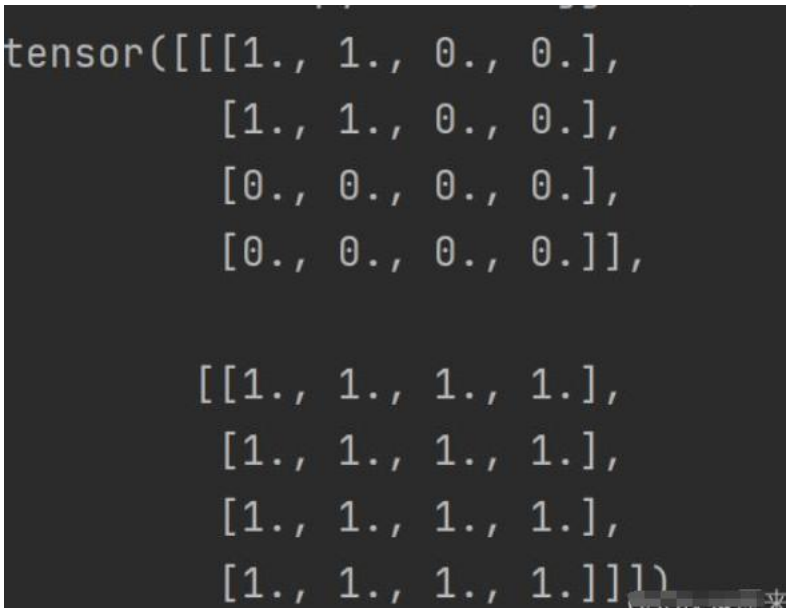


三、多头注意力

3.1 self mask

3.1.1 得到有效位置矩阵

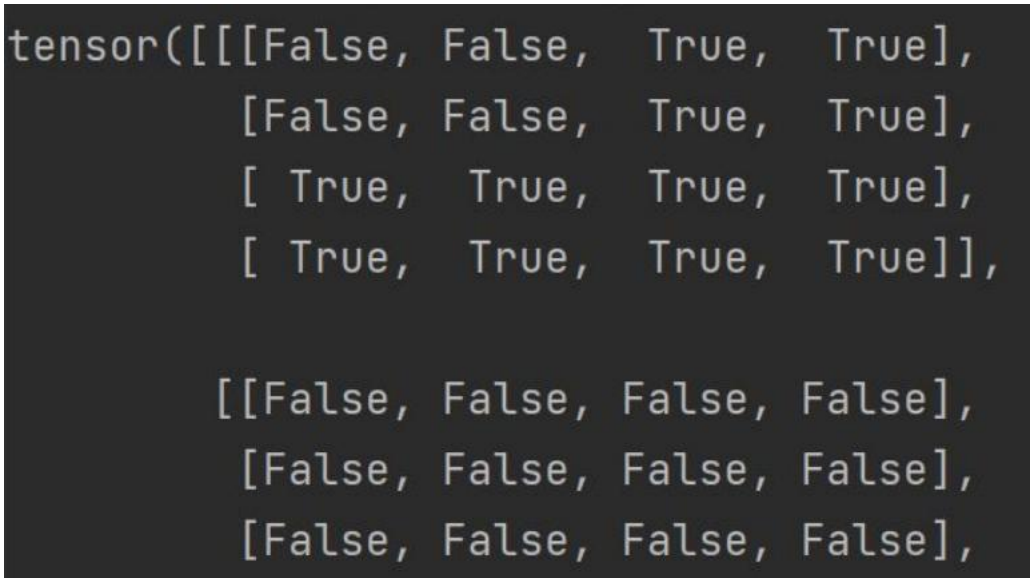
```
# 得到有效位置矩阵
vaild_encoder_pos = torch.unsqueeze(torch.cat([torch.unsqueeze(F.pad(torch.ones(L), (0, max(src_len) - L)), 0)for L in src_len]), 2)
valid_encoder_pos_matrix = torch.bmm(vaild_encoder_pos, vaild_encoder_pos.transpose(1, 2))
print(valid_encoder_pos_matrix)
```



3.1.2 得到无效位置矩阵

```
invalid_encoder_pos_matrix = 1-valid_encoder_pos_matrix
mask_encoder_self_attention = invalid_encoder_pos_matrix.to(torch.bool)
print(mask_encoder_self_attention)
```

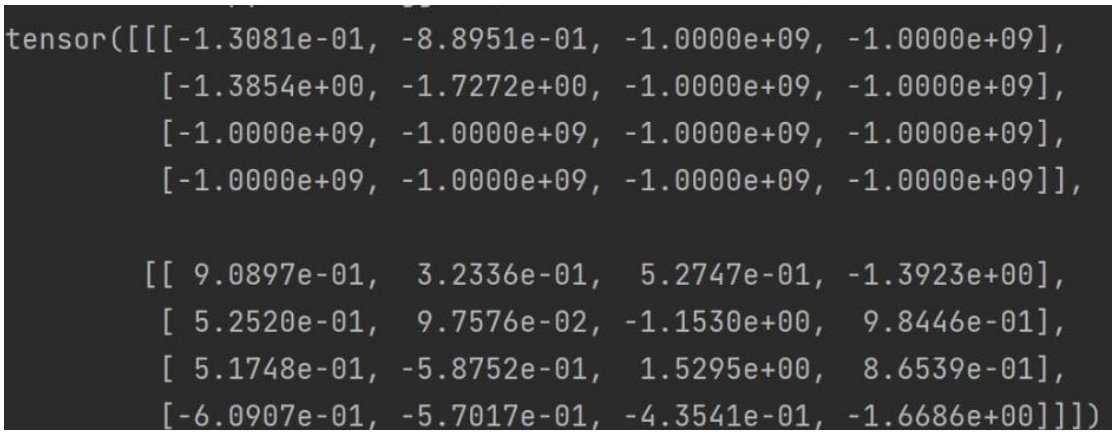
True 代表需要对该位置 mask



3.1.3 得到 mask 矩阵

用极小数填充需要被 mask 的位置

```
# 初始化 mask 矩阵
score = torch.randn(2, max(src_len), max(src_len))
# 用极小数填充
mask_score = score.masked_fill(mask_encoder_self_attention, -1e9)
print(mask_score)
```



算其 softmat

```
mask_score_softmax = F.softmax(mask_score)
print(mask_score_softmax)
```

可以看到，已经达到预期效果

